Experiment No. 8

Title: Implementation of problem based on Computational Geometry

Batch: B3 **Roll No: 16014225801** **Experiment No.:8**

Aim: Implementation of problem based on Computational Geometry

Resources needed: Text Editor, C/C++ IDE

Theory:

Rectangular grids of unit-spaced points (also called lattice points) are at the heart of any grid-based coordinate system. In general, there will be about one grid point per unit-area in the grid, because each grid point can be assigned to be the upper-right-hand corner of a different 1×1 empty rectangle. Thus the number of grid points within a given figure should give a pretty good approximation to the area of the figure. Pick's theorem gives an exact relation between the area of a lattice polygon (anon-intersecting figure whose vertices all lie on lattice points) and the number of lattice points on/in the polygon. Suppose there are I(P) lattice points inside of P and B(P) lattice points on the boundary of P. Then the area A(P) of P is given by

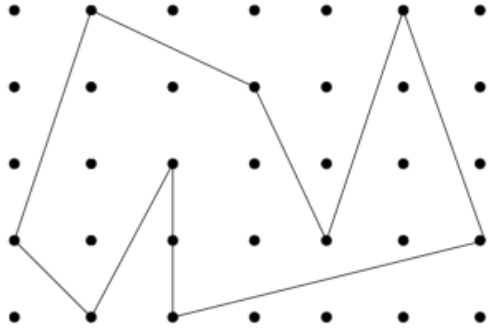
$$A(P)=I(P)+B(P)/2-1$$

as illustrated in Figure below. For example, consider a triangle defined by coordinates (x,1), (y,2), and (y+k,2). No matter what x, y, and k are there can be no interior points, because the three points lie on consecutive rows of the lattice. Lattice point (x,1) serves as the apex of the triangle, and there are k+ 1 lattice points on the boundary of the base. Thus I(P)=0,B(P)=k+ 2, and so the area is k/2, precisely what you get from the triangle area formula. As another example, consider a rectangle defined by corners (x1,y1) and (x2,y2). The number of boundary points is

$$B(P)=2|y2-y1+1|+2|x2-x1+1|-4 = 2(\Delta y-\Delta x)$$

with the 4-term to avoid double-counting the corners. The interior is the total number of points in or on the rectangle minus the boundary, giving

$$I(P)=(\Delta x+ 1)(\Delta y+1)-2(\Delta y-\Delta x)$$



Pick's theorem correctly computes the area of the rectangle as Δx Δy. Applying Pick's theorem requires counting lattice points accurately. This can in principle be done by exhaustive testing for small area polygons using functions that

(1) test whether a point lies on a line segment '

(2) test whether a point is inside or outside a polygon.

Example of Pick`s theorem:

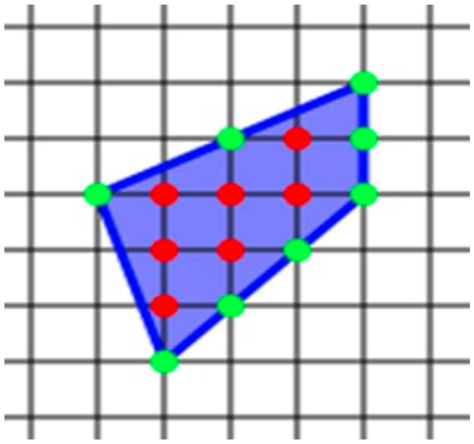
Pick's theorem is one of those theorems in mathematics which seems too simple to be true. Take any polygon and lay it on a lattice. (A lattice is a grid of points where every point has whole number (integer) coordinates.) According to Pick's Theorem all you need to do to find the area of a polygon is to count the points on the interior and on the boundary of the shape.

Pick's Theorem then states that:

$$Area = i + \frac{b}{2} - 1$$

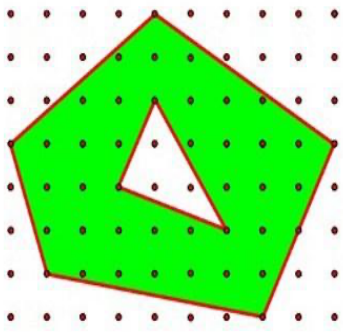
(i stands for the number of points in the interior of the shape, b stands for the number of points on the boundary of the shape.)

Take the polygon below as an example. In this polygon $i = 7$ and $b = 8$. According to Pick's Theorem, $Area = 7 + \frac{8}{2} - 1 = 10$ which is the correct area of the polygon.



Pick's Theorem does however only work for simple polygons – this means polygons which don't intersect themselves and don't have any holes, however it can be adapted to include holes.

Pick's Theorem for polygons with holes



To use Pick's Theorem on a shape like the one above you simply need to apply the theorem to the green shape without the hole and then subtract the area of the hole.

Area of Green shape (without hole) using Pick's Theorem: $38 + \frac{6}{2} - 1 = 40$

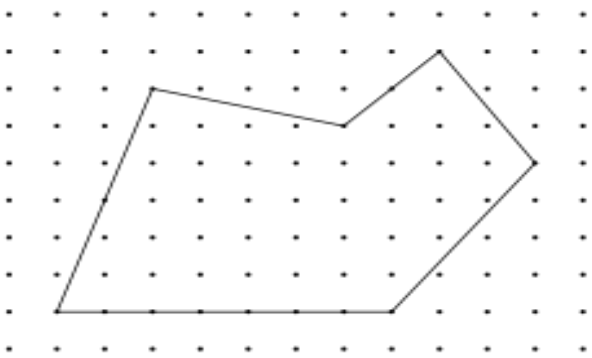
Area of the hole using Pick's Theorem: $3 + \frac{3}{2} - 1 = 3 + \frac{1}{2}$

Total Area of the shape = $40 - (3 + \frac{1}{2}) = 36 + \frac{1}{2}$

Activity:

Problem Statement: Green environment.

Mr. Cooper wants to plant trees on an island; however the island is not rectangular in shape (refer figure below). Using picks theorem he has to find a polygon area with vertices lying on the grid points and plant the trees strictly inside the grid points of this polygon with minimum distance from boundary of 1 unit. He has selected you for this job to write the program for above problem and state how many trees can be planted and what is area of the polygon mentioned below, distance between grid points is 1 unit.



Input format

A student has to consider input format as per their intelligence.

Output format

A student has to consider Output format as per their intelligence.

Constraints:

A student has to Design constraints as per your intelligence.

Solution:

Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

typedef long long ll;

struct Point {
    ll x, y;
};

struct Point p[1000];

ll gcd(ll a, ll b)
{
    if (b == 0)
        return a;

    return gcd(b, a % b);
}

double area(int n)
{
    ll sum = 0;

    for (int i = 0; i < n; i++)
    {
        int j = (i + 1) % n;

        sum += p[i].x * p[j].y - p[j].x * p[i].y;
    }

    return (double)llabs(sum) / 2.0;
}

ll boundary(int n)
{
    ll b = 0;

    for (int i = 0; i < n; i++)
    {
        int j = (i + 1) % n;

        ll dx = llabs(p[j].x - p[i].x);
        ll dy = llabs(p[j].y - p[i].y);

        b += gcd(dx, dy);
    }

    return b;
```

```
}

int inside(double x, double y, int n)
{
    int c = 0;

    for (int i = 0, j = n - 1; i < n; j = i++)
    {
        if ((p[i].y > y) != (p[j].y > y))
        {
            double t = (p[j].x - p[i].x) *
                       (y - p[i].y) /
                       (p[j].y - p[i].y) + p[i].x;

            if (x < t)
                c = !c;
        }
    }

    return c;
}

double distance(double x, double y, struct Point a, struct Point b)
{
    double dx = b.x - a.x;
    double dy = b.y - a.y;

    double t = ((x - a.x) * dx + (y - a.y) * dy) /
               (dx * dx + dy * dy);

    if (t < 0)
        t = 0;

    if (t > 1)
        t = 1;

    double px = a.x + t * dx;
    double py = a.y + t * dy;

    return hypot(x - px, y - py);
}

int valid(double x, double y, int n)
{
    for (int i = 0; i < n; i++)
    {
        int j = (i + 1) % n;

        if (distance(x, y, p[i], p[j]) < 1.0 - 1e-9)
            return 0;
    }
```

```
    return 1;
}

int main()
{
    int n;
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        scanf("%lld %lld", &p[i].x, &p[i].y);
    }

    double a = area(n);
    ll b = boundary(n);

    ll interior = (ll)(a - b / 2.0 + 1);

    ll trees = 0;

    ll minx = p[0].x, maxx = p[0].x;
    ll miny = p[0].y, maxy = p[0].y;

    for (int i = 1; i < n; i++)
    {
        if (p[i].x < minx)
            minx = p[i].x;

        if (p[i].x > maxx)
            maxx = p[i].x;

        if (p[i].y < miny)
            miny = p[i].y;

        if (p[i].y > maxy)
            maxy = p[i].y;
    }

    for (ll x = minx; x <= maxx; x++)
    {
        for (ll y = miny; y <= maxy; y++)
        {
            if (inside(x, y, n) && valid(x, y, n))
                trees++;
        }
    }

    printf("Area = %.2f\n", a);
    printf("Interior points = %lld\n", interior);
    printf("Trees = %lld\n", trees);
    return 0;
}
```

Output:

```
(base) bhumit@bhumits-MacBook-Air exp8 % "/Users/bhumit/Desktop/Sem
5/cpl/code/exp8/exp8"
4
0 0
6 0
6 4
0 4
Area = 24.00
Interior points = 15
Trees = 15
```

Heatwave Use Case Implementation:

This program can be implemented by representing the island as a polygon using grid coordinates. The Shoelace Formula calculates its area, while Pick's Theorem determines the number of interior grid points. The program checks each interior point and calculates its distance from the polygon's boundary. Points at least one unit away are considered suitable for planting trees. Using C, functions can be created for area calculation, boundary points, interior checking, and tree counting. Finally, the program displays the polygon's area and the maximum number of trees that can be planted, helping estimate planting capacity within the island.

Conclusion:

In conclusion, this program uses Pick's Theorem and geometric calculations to determine polygon area and identify suitable interior grid points for planting trees efficiently and we learnt and completed the experiment

---

References:

1. Steven S. Skiena Miguel A. Revilla,"Programming challenges, The Programming Contest Training Manual", Springer, 2006
2. T.H. Coreman ,C.E. Leiserson,R.L. Rivest, and C. Stein, " Introduction to algorithms", 3rd Edition 2009, Prentice Hall India Publication
3. Antti Laaksonen, "Guide to Competitive Programming",Springer,2018